

Learned Encoder-Decoder Models for Diffusion-Like Image Encryption: A Machine-Learning Approach Based on Stochastic Processes

Samuel Cavazos
Alshival.AI

March 23, 2026

Abstract

This paper documents a research prototype for learned image encryption based on a fixed stochastic forward process and paired neural endpoint maps. Starting from RGBA sprite images, we apply a diffusion-style corruption process to RGB channels while preserving alpha, then train one U-Net-like network to emulate the forward transform and a second network to learn its inverse. The design was motivated by two practical observations: first, early experiments with deterministic non-linear tensor morphisms were difficult to generalize across images; second, stochastic corruption offers analytic control over signal attenuation, direct compatibility with one-step denoising ideas from consistency models, and a tunable bridge between structure preservation and information destruction.

The resulting system is successful as a research instrument: it trains stably, produces reproducible paired data, and learns low-error endpoint regressors on the current dataset. As a toy example, one can view the workflow as a machine-learning version of “Who’s That Pokemon?”: Mew is mapped to an obscured encoded representation, transmitted in that form, and then reconstructed by a decoder at the receiving end, with the original sprite reused below as the toy decryption target. It is not yet a cryptosystem. For the default schedule used here, the pre-clipping signal coefficient after 1000 steps is only $\alpha_T = 5.071e - 12$, meaning that the RGB signal energy is effectively annihilated before clipping and quantization. Empirically, the final encoded RGB values have correlation -0.0006 with the source image and 65.7% of encoded RGB channel values lie exactly at 0 or 1. The mathematical and empirical conclusion is that the current process is excellent for studying learned reconstruction under severe stochastic corruption, but exact reversibility and cryptographic security require stronger design constraints than those implemented in this prototype.



1 Introduction

This repository explores whether a machine-learning model pair can act like an encoder/decoder system for images after a deliberately destructive forward transform. The motivating application is not ordinary image compression; it is the stronger ambition of learning an *encryption-like* transport mechanism in which a sender applies a difficult-to-interpret transformation and a receiver learns the inverse.

One long-term application class is privacy-sensitive image transport. In a future secure version of this framework, a sender-side model could encode a sensitive image, such as a radiology scan, pathology slide, or other healthcare image, into a transit representation that is difficult to interpret directly, send that representation over a network, and then use a receiver-side model to decode it after arrival. If that workflow were paired with keyed randomness and exact authorized recovery, it could complement conventional transport protections by reducing exposure of directly viewable patient content while images move between institutions or services. The current repository does not satisfy the security, reversibility, or regulatory requirements for protected health information; here the healthcare example is a motivating application target, not a claim of deployment readiness.

As a toy example, the same transport idea can be described with a single sprite:

$$\text{Mew (Original)} \xrightarrow{\text{encoder}} \text{Mew (Encrypted, in transit)} \xrightarrow{\text{decoder}} \text{Mew (Decrypted)}. \quad (1)$$

In that conceptual pipeline, “encryption in transit” refers to the middle stage, where only the encoded representation is transmitted between endpoints. The original image remains at the sender, and the decrypted reconstruction appears only after the encoded payload reaches the receiver. In the present paper this Mew example is illustrative only: it explains the intended sender/network/receiver workflow, not a claim that the current prototype already provides cryptographically secure transit protection.

Our early internal experiments focused on deterministic non-linear tensor morphisms: handcrafted non-linear spatial and channel transforms designed to scramble image tensors without explicit randomness. In practice, these morphisms were difficult to generalize. They often behaved like brittle texture-specific warps: they could fit seen patterns, but they did not produce a clean family of transformations with obvious scaling laws or analytically controlled information loss. That experience motivated a shift toward a stochastic process.

The stochastic choice was informed by diffusion and consistency-model literature. Diffusion models are attractive because the forward corruption process is mathematically tractable, the signal-to-noise tradeoff is tunable through a schedule, and a learned network can target endpoint recovery instead of inverting an arbitrary handcrafted morphism [1, 3, 4]. Our prototype adopts that philosophy in a simplified supervised form: generate paired samples with a fixed stochastic process, then learn direct maps for the forward and reverse endpoints.

Contributions. This paper makes four concrete contributions:

1. It formalizes the current stochastic forward transform and derives its closed-form endpoint distribution before clipping.
2. It explains why a stochastic path was chosen over deterministic non-linear tensor morphisms for this line of work.
3. It reports the empirical behavior of the current repository implementation, including quantitative distortion statistics, endpoint training curves, and qualitative reconstructions.
4. It identifies the precise gap between the current prototype and a true cryptographic system.

2 Background and Motivation

2.1 Why stochastic corruption is attractive

Let $x_0 \in [0, 1]^{H \times W \times C}$ denote an image. A deterministic tensor morphism seeks a map

$$z = g(x_0), \quad (2)$$

where g is intentionally hard to interpret yet still invertible or approximately invertible. The problem is that once g becomes sufficiently non-linear to hide semantics, it typically becomes difficult to characterize globally. Local Jacobians can vary sharply across the image manifold, and small changes in spatial structure may produce qualitatively different outputs. In our exploratory work this made generalization brittle.

By contrast, a stochastic process defines a family of transforms indexed by noise variables:

$$z = T_\omega(x_0), \quad (3)$$

where ω denotes a random seed or noise path. This has three advantages.

First, the endpoint distribution can often be analyzed exactly or approximately. Second, the process difficulty can be tuned continuously by a schedule rather than redesigned from scratch. Third, the same corrupted endpoint can be paired with direct learned maps, echoing the endpoint-consistency view that motivates one-step generation and denoising in consistency models [4].

2.2 Relation to consistency models

Consistency models learn functions that map points on a noise path toward a shared consistent output, allowing high-quality generation in very few steps [4]. Our method is not a consistency model in the strict sense: we do not train across multiple times with a consistency objective, nor do we solve the probability-flow ODE used in that work. The connection is conceptual rather than identical. The consistency-model literature encouraged us to prefer a stochastic path with analyzable marginals and to investigate whether direct endpoint maps can work without iterative reverse sampling.

3 Method

3.1 Data domain

The current repository uses 809 Pokemon sprite pairs as a controlled RGBA image dataset. Let

$$x_0 = (r_0, g_0, b_0, a_0) \in [0, 1]^{H \times W \times 4}. \quad (4)$$

The RGB channels are treated as the signal to corrupt; the alpha channel is preserved exactly in the current implementation.

3.2 Forward process

Define $y_t \in \mathbb{R}^{H \times W \times 3}$ as the RGB state at step t , with $y_0 = (r_0, g_0, b_0)$. The forward process implemented in the repository is

$$y_t = \sqrt{1 - \beta_t} y_{t-1} + \sqrt{\beta_t} \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, I), \quad (5)$$

for $t = 1, \dots, T$, where $T = 1000$ and the schedule is linearly interpolated from $\beta_1 = 0.0005$ to $\beta_T = 0.1$.

The stored encoded image is then

$$z = (Q(\text{clip}(y_T, 0, 1)), a_0), \quad (6)$$

where clip clamps each RGB value to $[0, 1]$ and Q denotes 8-bit PNG quantization.

Implementation note. For reproducible paired supervision, the current public prototype uses a fixed pseudorandom realization during dataset generation. In other words, the *family* is stochastic, but the released dataset corresponds to one realized corruption path. This is useful for supervised learning and reproducibility, but it is weaker than a deployment setting with keyed per-message randomness.

3.3 Learned endpoint maps

Two separate neural networks are trained:

$$f_{\theta_e} : x_0 \mapsto z, \quad (7)$$

$$f_{\theta_d} : z \mapsto x_0. \quad (8)$$

Both are instantiated as the same compact U-Net-like CNN with 4,335,508 trainable parameters, built from three downsampling blocks, a bottleneck, and three decoder blocks with skip connections following the U-Net design pattern [2].

Training uses an L_1 objective:

$$\mathcal{L}_{\text{enc}}(\theta_e) = \frac{1}{N} \sum_{i=1}^N \left\| f_{\theta_e}(x_0^{(i)}) - z^{(i)} \right\|_1, \quad (9)$$

$$\mathcal{L}_{\text{dec}}(\theta_d) = \frac{1}{N} \sum_{i=1}^N \left\| f_{\theta_d}(z^{(i)}) - x_0^{(i)} \right\|_1. \quad (10)$$

The repository reports dataset mean absolute error (MAE), which is the elementwise absolute error averaged over every pixel and channel.

4 Mathematical Analysis

4.1 Closed-form endpoint distribution

Equation (5) is a discrete-time linear Gaussian process on RGB pixels. Define

$$\alpha_t = \prod_{s=1}^t \sqrt{1 - \beta_s}. \quad (11)$$

Proposition 1. *Conditioned on y_0 , the pre-clipping endpoint distribution is*

$$y_t \mid y_0 \sim \mathcal{N}(\alpha_t y_0, (1 - \alpha_t^2)I). \quad (12)$$

Proof. Unrolling Equation (5) yields

$$y_t = \alpha_t y_0 + \sum_{k=1}^t \left(\sqrt{\beta_k} \prod_{s=k+1}^t \sqrt{1 - \beta_s} \right) \varepsilon_k. \quad (13)$$

The mean conditioned on y_0 is therefore $\alpha_t y_0$. Since the ε_k are independent standard Gaussians, the covariance is the sum of the squared coefficients:

$$\sum_{k=1}^t \beta_k \prod_{s=k+1}^t (1 - \beta_s) = 1 - \prod_{s=1}^t (1 - \beta_s) = 1 - \alpha_t^2. \quad (14)$$

Thus $y_t \mid y_0$ is Gaussian with the claimed mean and covariance. $\square$

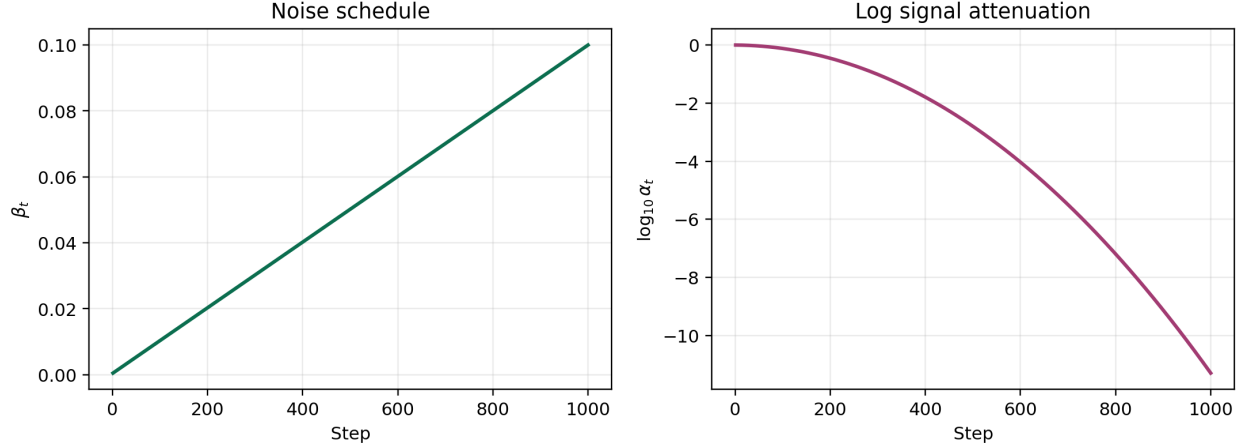


Figure 1: The default corruption schedule used in the repository. Left: the linear β_t schedule. Right: the log attenuation coefficient $\log_{10} \alpha_t$. By the final step, the RGB signal contribution is effectively annihilated.

4.2 Why the chosen schedule is so destructive

For the default schedule, the final attenuation coefficient is

$$\alpha_T = 5.071e - 12, \quad \alpha_T^2 = 2.571e - 23. \quad (15)$$

This means the linear signal energy surviving in RGB at step T is effectively zero before clipping. The forward process therefore behaves almost like pure Gaussian noise at the endpoint, after which clipping and 8-bit quantization collapse many distinct continuous states into the same stored PNG values.

Figure 1 makes this explicit. The linear schedule increases noise steadily, while the surviving signal coefficient decays superlinearly in log scale.

4.3 Implications for invertibility

The current forward operator is not injective once clipping and quantization are applied. Even if two distinct source images $x_0 \neq x'_0$ induce different continuous distributions before quantization, the map in Equation (6) collapses broad regions of RGB space to the same 8-bit values. Exact inversion is therefore impossible in general.

This is the central mathematical distinction between our prototype and a cryptosystem. A cryptosystem can be computationally hard to invert without a key while still being exactly invertible with the key. The present process is instead *information-destroying*. Recovery must therefore come from learned image priors and dataset regularity, not from formal reversibility.

4.4 Role of alpha preservation

The alpha channel is preserved exactly in the dataset generator, so the raw alpha-channel MAE between original and encoded images is 0.000000. This matters for interpretation. A zero-error alpha channel lowers aggregate MAE while leaving RGB corruption unchanged. Consequently, RGB-only MAE is the more informative statistic when discussing semantic recovery.

5 Experimental Setup

The public experiment uses the repository’s current configuration:

- Dataset size: 809 paired images.
- Image resolution during training: 128×128 RGBA.
- Forward schedule: 1000 steps, linear β_t from 0.0005 to 0.1.
- Architecture: U-Net-like CNN with 4,335,508 trainable parameters.
- Objective: elementwise L_1 loss.
- Training logs used here: 20-epoch encoder and decoder runs from the repository’s quick-eval model directory.

These are training-set results, not holdout results. They are useful for understanding whether the endpoint maps can fit the current paired transformation, but they should not be interpreted as evidence of security or strong out-of-distribution generalization.

6 Results

6.1 Forward-process statistics

The encoded RGB endpoint is heavily saturated and nearly decorrelated from the source RGB. Across the current dataset:

- 65.7% of encoded RGB values are exactly 0 or 1 after clipping and quantization.
- The flattened RGB correlation between source and encoded images is -0.0006.
- The alpha channel is unchanged.

This validates the intended behavior of the stochastic transform: the visible endpoint is hard to interpret directly. It also explains why exact inversion is mathematically impossible in the current design.

6.2 Training behavior

Both endpoint networks train smoothly over the first 20 epochs, with monotone decreases in train L_1 and dataset MAE. Figure 2 shows that the decoder converges slightly better than the encoder on this dataset.

6.3 Quantitative endpoint accuracy

Table 1 summarizes the main quantitative results. The direct baseline compares the original image to the encoded image without a learned model. Both learned models improve substantially over that baseline.

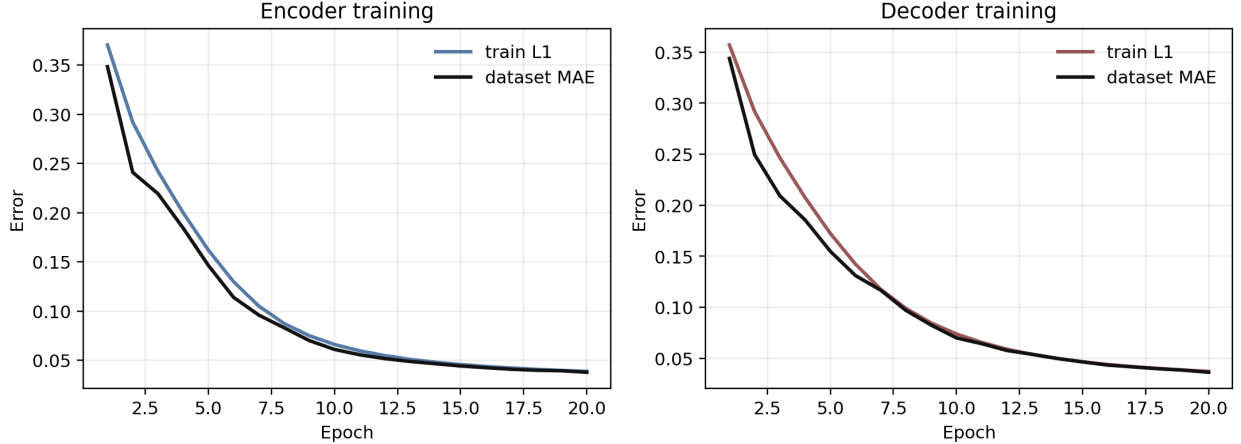


Figure 2: Training dynamics for the encoder and decoder endpoint networks. Both optimize stably, supporting the claim that the stochastic pairing setup is learnable even with a relatively small convolutional model.

Metric	Direct baseline	Encoder	Decoder
All-channel MAE	0.04963996	0.03785820	0.03638585
RGB-only MAE	0.06618662	0.04731516	0.04471758
RGB PSNR (dB)	14.33	18.47	19.86

Table 1: Dataset-level endpoint metrics on the current paired dataset. All-channel MAE is lower than RGB-only MAE because alpha is preserved exactly by the forward generator.

6.4 Qualitative behavior

Figure 3 illustrates the current visual behavior. The encoded targets are visually noisy and semantically obscured. The learned encoder captures the silhouette well but does not reproduce the target’s multicolored endpoint distribution exactly. The learned decoder reconstructs shape reliably, but the recovered images remain muted and often collapse toward silhouette-first solutions.

Figure 4 reinforces the same conclusion. The encoded RGB distribution is sharply concentrated at clipped values, and the learned models reduce error against the direct baseline but do not recover a truly invertible channel.

7 Discussion

7.1 What succeeded

The stochastic reformulation solved several problems that appeared in the earlier deterministic-morphism stage of this project.

First, it gave the experiment a mathematically transparent forward path. Second, it produced a stable supervised learning problem with direct paired targets. Third, it enabled one-step learned endpoint maps that train cleanly with a simple convolutional architecture. These are meaningful successes for a research prototype.

7.2 What failed, or remains unresolved

Three limitations are structural rather than incidental.

The process is not reversible. Because clipping and quantization destroy information, the decoder is solving an image-reconstruction problem under a strong prior, not decrypting a formally invertible ciphertext.

The current implementation leaks alpha. Preserving alpha means one quarter of the channels are already solved. This lowers aggregate error and exposes silhouette information directly.

The public prototype fixes a single realized corruption path. That choice is appropriate for reproducible training pairs, but it is not a secure keyed scheme. A production-grade system would need secret per-message randomness or a keyed pseudorandom generator driving the corruption schedule.

7.3 Why the stochastic choice was still the right research move

Despite those limitations, the stochastic choice was justified. It gave us an analyzable corruption operator, a well-conditioned training target, and a direct way to study the gap between *learned reconstruction under noise* and *cryptographic reversibility*. Deterministic non-linear tensor morphisms did not provide that level of control or transparency in our exploratory work.

8 Future Work

Several directions follow naturally from the present findings:

1. Replace the fixed realized noise path with keyed per-sample stochasticity.
2. Remove or separately encrypt alpha to avoid silhouette leakage.
3. Study invertible or volume-preserving transforms when exact recoverability is required.
4. Add holdout and cross-domain evaluations to distinguish memorization, generalization, and true prior-based reconstruction.
5. Explore multi-time training objectives inspired more directly by consistency models instead of only endpoint supervision.

9 Conclusion

This project demonstrates that a diffusion-inspired stochastic corruption process is a practical and mathematically legible replacement for brittle deterministic tensor morphisms in early machine-learning-for-encryption research. The resulting endpoint networks do learn the paired transform on the present dataset, and the training behavior is stable. At the same time, the mathematics makes clear why the current system is not yet encryption in the cryptographic sense: the forward process nearly eliminates RGB signal before clipping, the stored endpoint is not injective, and recovery depends on learned priors rather than exact reversibility.

That is still a useful result. The prototype succeeds as a research platform for studying stochastic transport, one-step learned reconstruction, and the boundary between image priors and secure invertible encoding. The next stage should preserve the mathematical clarity of the stochastic framework while tightening the design toward keyed randomness and formally reversible transforms.

References

- [1] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html>.
- [2] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, pages 234–241. Springer, 2015. URL https://link.springer.com/chapter/10.1007/978-3-319-24574-4_28.
- [3] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=PxTIG12RRHS>.
- [4] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 32211–32252, 2023. URL <https://proceedings.mlr.press/v202/song23a.html>.

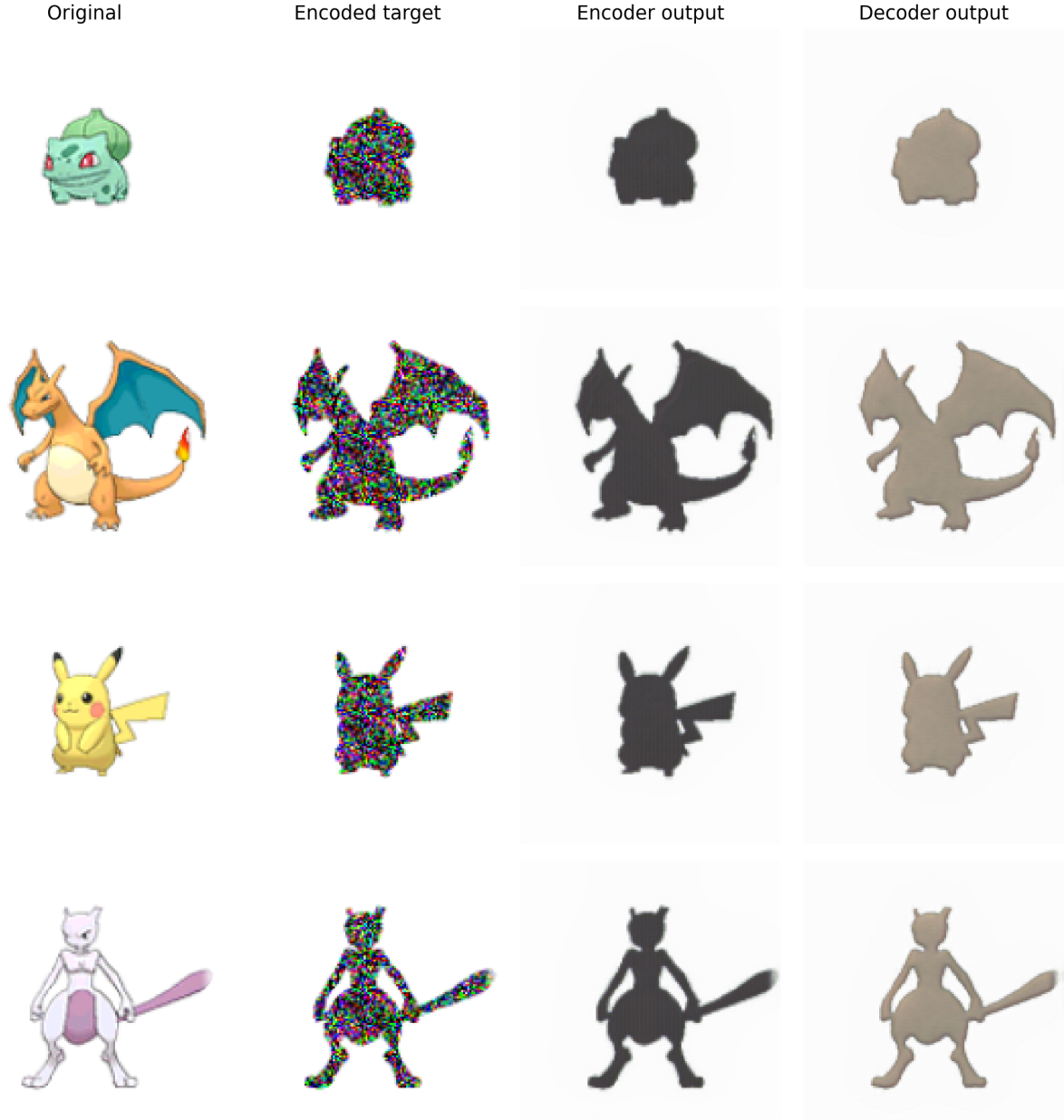


Figure 3: Qualitative examples from the current repository checkpoints: bulbasaur, charizard, pikachu, and mewtwo. The decoder recovers recognizable geometry, but color fidelity remains limited, which is consistent with the severe endpoint information loss predicted by the theory.

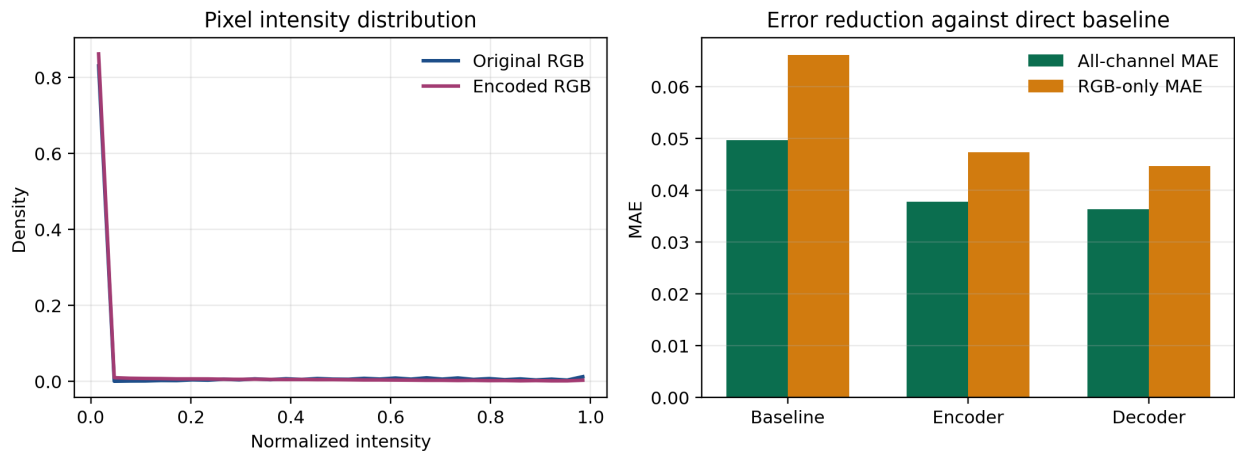


Figure 4: Left: original and encoded RGB intensity distributions. Right: error reduction relative to the direct baseline. The current decoder improves over the baseline, but the improvement is incremental rather than evidence of full reversibility.